

TP 5 : calculs avec les polynômes

ZFAI

Les questions marquées d'une $\star$ sont des questions théoriques. Elles peuvent être abordées plus tard, après le TP.

L'objectif est d'implémenter différents algorithmes nous permettant de faire des calculs avec des polynômes. On rappelle qu'un polynôme est un objet qui s'écrit sous la forme :

$$P(X) = a_0 + a_1X + a_2X^2 + \cdots + a_nX^n.$$

Ainsi, on peut représenter le polynôme P en python à l'aide de la liste de ses coefficients :

$$L = [a_0, a_1, a_2, \cdots, a_n].$$

Sur les polynômes, il existe différentes opérations élémentaires :

1. la somme,
2. le produit,
3. la dérivation,
4. le calcul de $P(a)$ pour un réel a .

Une question naturelle est alors d'implanter ces opérations en Python, pour effectuer des calculs sur les polynômes à l'aide de l'ordinateur.

À FAIRE

Exercice 1. Un premier problème concerne l'unicité de la représentation. En effet, deux listes différentes peuvent correspondre à un même polynôme. Par exemple, $L = [0, 0, 1, 0]$ et $M = [0, 0, 1]$ correspondent toutes les deux au polynôme X^2 . De même, le polynôme nul peut être représenté par $[]$ ou par $[0]$.

1. Écrire une fonction `polEgaux(L,M)` qui retourne `True` si L et M correspondent au même polynôme et `False` sinon.
2. Écrire une fonction `reduire(L)` qui supprime de L les 0 "inutiles" pour représenter P .
3. Une notion fondamentale sur les polynômes est le degré. Écrire une fonction `degre(P)` qui retourne le degré du polynôme P . Par convention, le degré du polynôme nul sera -1 .

Remarque. En mathématiques, le degré du polynôme nul est $-\infty$. Mais il est plus facile, pour l'implémentation de le représenter ici par -1 .

Correction

1. Voici une fonction possible :

```
def polEgaux(L,M):
    l=len(L)
    m=len(M)
    for i in range(0,min(l,m)):
        if L[i]!=M[i]:
            return False
    for j in range(min(l,m),max(l,m)):
        if l>m and L[j]!=0:
            return False
        elif m>l and M[j]!=0:
            return False
    return True
```

2. Voici une fonction possible :

```
def reduire(L):
    if len(L)>0 :
        n=len(L)-1
```

```

        while (n>=0 and L[n]==0) :
            L.pop()
            n=n-1
    return L

```

3. Voici une fonction possible :

```

def degre(P):
    n=len(P)-1
    while (n>=0 and P[n]==0) :
        n=n-1
    return n

```

Exercice 2. Écrire une fonction `derive(P)` qui retourne une liste représentant P' .

Correction

Voici une fonction possible :

```

def derive(P) :
    L=[0 for i in range(len(P)-1)]
    for i in range(len(P)-1) :
        L[i]=(i+1)*P[i+1]
    return L

```

Exercice 3. Écrire une fonction `somme(P,Q)` qui prend en argument deux listes représentant des polynômes et qui retourne la somme des deux.

Correction

Voici une fonction possible :

```

def somme(P,Q):
    p=len(P)
    q=len(Q)
    L=[]
    for i in range(0,min(p,q)):
        L.append(P[i]+Q[i])
    for j in range (min(p,q),max(p,q)):
        if p>q :
            L.append(P[j])
        else :
            L.append(Q[j])
    return L

```

Exercice 4 (Méthode d'évaluation en un point). 1. On rappelle que pour évaluer en un point, on remplace X par valeur a . Écrire une fonction `evaluation(P,a)` qui retourne la valeur $P(a)$, le calcul étant effectué de manière naïve.

- (★) Déterminer la complexité en nombre de multiplication de votre fonction.
- Une des méthodes pour évaluer de manière efficace un polynôme en un point est appelée méthode de Horner. Elle consiste à remarquer le fait suivant :

$$a_0 + a_1X + a_2X^2 + a_nX^n = a_0 + X(a_1 + X(a_2 + \cdots + X(a_{n-1} + a_nX))) \cdots)$$

Pour calculer $P(a)$, on effectue alors les affectations suivantes :

- (a) $S = a_n$, (b) $S = a_n a$, (c) $S = S + a_{n-1}$, (d) $S = Sa$,
 (e) $S = S + a_{n-2}$, (f) $\cdots$ (g) $S = S + a_1$, (h) $S = Sa$, (i) $S = S + a_0$.

Par exemple, pour évaluer le polynôme $X^3 + 5X^2 + 2X + 1$ en 2, on a donc successivement : 1,2,7,14,16,32,33. On a donc $P(2) = 33$.

Écrire une fonction `Horner(P,a)` qui calcule $P(a)$ à l'aide de la méthode de Horner.

4. (★) Déterminer la complexité de la méthode de Horner en nombre de multiplications.
5. (Optionnel) Proposer un code qui permet de comparer informatiquement ces deux méthodes d'évaluation.

Correction

1. Voici une fonction possible :

```
def evaluation(P,a) :
    S=0
    for i in range(len(P)) :
        S+=P[i]*a**i
    return S
```

2. Cela dépend de la façon dont est implémenté l'exponentiation (**). Dans le cas où l'implémentation est naïve, on a alors comme complexité $O(\sum_{i=1}^n i)$, c'est-à-dire en $O(n^2)$. En python, en réalité l'exponentiation est rapide. Donc $a ** i$ nécessite $O(\log(i))$ multiplications. Ainsi, la complexité est en $O(\sum_{i=1}^n \log(i))$, c'est-à-dire en $O(n \log(n))$.
3. Voici une fonction possible :

```
def Horner(P,a) :
    if len(P)>0 :
        n=len(P)-1
        S=0
        for i in range(n,0,-1) :
            S=S+P[i]
            S=S*a
        S=S+P[0]
        return S
    else :
        return 0
```

4. Dans la méthode de Horner, on a une boucle for avec i qui varie de 1 à n . Pour chaque i , on effectue 1 multiplication. Ainsi, la complexité est en $O(n)$ multiplications.
5. Voici un exemple pour effectuer un test de comparaisons.

```
import random as rd
import time
def test(n) :
    print("Pour n=", n)
    P = [rd.randint(1,100) for i in range(n)]
    A = [rd.randint(1,100) for _ in range(n)]
    t0 = time.time()
    [evaluation(P,a) for a in A]
    t1 = time.time()
    fin = t1-t0
    print("test evaluation naive :", fin)
    t0 = time.time()
    [Horner(P,a) for a in A]
    t1 = time.time()
    fin = t1 - t0
    print("test evaluation Horner :", fin)

for i in range(1,5) :
    test(10**i)
```

- Exercice 5** (Multiplication naïve). 1. Écrire une fonction `produit(P,Q)` prenant en argument deux listes représentant des polynômes et qui renvoie la liste représentant le produit correspondant.
2. (★) Expliciter la complexité en nombre de produits de nombres.

Correction

1. Voici une fonction possible :

```
def produit(P,Q) :
    L = [0 for _ in range(len(P)+len(Q)-1)]
    for k in range(len(P)) :
        for l in range(len(Q)) :
            L[k+l] += P[k]*Q[l]
    return L
```

2. On compte directement le nombre de multiplications. On a : $\sum_{0 \leq k < \text{len}(P)} \sum_{0 \leq l < \text{len}(Q)} 1$. Ce qui donne directement $\text{len}(P)\text{len}(Q)$ multiplications.

Exercice 6 (Division naïve). 1. Écrire une fonction `division(P,Q)` prenant en argument deux listes représentant des polynômes et qui renvoie la liste représentant le reste du polynôme P par Q .

2. (★) Expliciter la complexité en nombre de produits et divisions de nombres.

Correction

1. Voici une fonction possible :

```
def division(P,Q) :
    n = degre(P)
    m = degre(Q)
    if m == -1 :
        raise ValueError
    if m > n :
        return P
    P1 = reduire(P)
    A = [0 for _ in range(n-m)] + [(-P[n]/Q[m])*Q[i] for i in range(m)] + [-P[n]]
    nouv_P = somme(P,A)
    return division(nouv_P,Q)
```

2. Dans le cas où P a un degré inférieur strict à Q , il y a 0 multiplication. Dans le cas où le degré de P (noté n) est supérieur au degré de Q (noté m), on effectue $2m$ opérations de ce type. Le nouveau polynôme a alors un degré inférieur à $n - 1$ et on recommence. On en déduit que le nombre d'opérations est en $O(\text{len}(Q)(\text{len}(P) - \text{len}(Q)))$.

Pour les plus rapides

Exercice 7. On s'intéresse à un algorithme de multiplications de polynômes, appelé algorithme de Karatsuba.

Cet algorithme est basé sur la remarque suivante. Pour deux polynômes $P = P_0 + X^n P_1$, $Q = Q_0 + X^n Q_1$ avec P_0, P_1, Q_0, Q_1 de degré au plus $n - 1$, on a

$$PQ = P_0Q_0 + ((P_0 + P_1)(Q_0 + Q_1) - P_0Q_0 - P_1Q_1)X^n + X^{2n}P_1Q_1 \quad (1)$$

1. En exploitant cette égalité et en procédant de manière récursive, écrire une nouvelle fonction `produit_rapide(P,Q)` qui renvoie le produit de deux polynômes. On supposera pour simplifier que les listes représentant P et Q ont une taille qui est une puissance de 2.
Remarque. Pour des polynômes de petits degrés, on remarque qu'en pratique le produit naïf suffit et est même plus efficace. Ainsi, on peut proposer une fonction qui procède de manière naïve pour des petits degrés (par exemple 32) et qui utilise l'algorithme Karatsuba pour des degrés plus grand.
2. (★) Expliquer comment se ramener au cas des puissances de 2 dans le cas général.
3. (★) Proposer un code qui permet de comparer informatiquement ces deux méthodes d'évaluation.
4. (★) Déterminer la complexité de votre nouvelle fonction.

Correction

1. Voici un exemple de fonction :

```
def produit_rapide(P,Q) :
    if len(P) <= 32 or len(Q) <= 32 :
        return produit1(P,Q)
```

```

n = len(P)
P0 = P[:n//2]
P1 = P[n//2:]
Q0 = Q[:n//2]
Q1 = Q[n//2:]
P0Q0 = produit_rapide(P0,Q0)
P1Q1 = produit_rapide(P1,Q1)
produit_milieu = produit_rapide(somme(P0,P1),somme(Q0,Q1))
terme_central = [0]*(n//2) + \
[produit_milieu[i]-P0Q0[i]-P1Q1[i] for i in range(len(produit_milieu))]
return somme(P0Q0,somme(terme_central,[0]*(n)+P1Q1))

```

2. Pour se ramener au cas des puissances de 2, il suffit de rajouter des 0 en fin de liste jusqu'à ce que l'on ait des tailles qui sont des puissances de 2.
3. Voici un exemple de fonction test possible :

```

def test_produit(n) :
    print("Pour n=", 2**n)
    P = [rd.randint(1,100) for i in range(2**n)]
    A = [rd.randint(1,100) for _ in range(2**n)]
    t0 = time.time()
    produit1(P,A)
    t1 = time.time()
    fin = t1-t0
    print("test evaluation naive :", fin)
    t0 = time.time()
    produit_rapide(P,A)
    t1 = time.time()
    fin = t1 - t0
    print("test evaluation karatsuba :", fin)

```

Par exemple, on obtient avec $n = 13$:

```

Pour n= 8192
test evaluation naive : 6.692143201828003
test evaluation karatsuba : 0.6635174751281738

```

4. On se place dans le où la taille est une puissance de 2. Il existe $K \geq 1$ tel que :

$$C(2^p) \leq 3C(2^{p-1}) + K2^p.$$

En itérant une deuxième fois, on trouve :

$$C(2^p) \leq 3^2 C(2^{p-2}) + 3K2^{p-1} + K2^p$$

Par récurrence et en constatant que $C(1) = 1 \leq K$, on obtient :

$$C(2^p) \leq K \sum_{j=0}^p 3^j 2^{p-j}$$

On factorise par 2^p . On a alors :

$$C(2^p) \leq K2^p \frac{1 - (\frac{3}{2})^{p+1}}{1 - \frac{3}{2}}$$

Donc :

$$C(2^p) \leq K3^{p+1}$$

Ainsi, $C(2^p) = O(3^p)$. En revenant sur $n = 2^p$. On en déduit que :

$$C(n) = O(n^{\frac{\ln(3)}{\ln(2)}})$$

On peut remarquer que dans le cas où la taille n'est pas une puissance de deux, on encadre l'entier n par deux puissances de 2 successives. Ces deux puissances donnent un même ordre de grandeur pour la complexité. Ainsi, par encadrement, il en est de même de $C(n)$.

Exercice 8 (Division quotient). Écrire une fonction `quotient_reste(P,Q)` prenant en arguments deux listes représentant des polynômes et qui renvoie un couple de listes (S, R) qui représente des polynômes vérifiant : $P = SQ + R$, où R est le reste et S le quotient.

Correction

Voici le code correspondant

```
def quotient_reste(P,Q) :
    n = degre(P)
    m = degre(Q)
    if m == -1 :
        raise ValueError
    if n < m :
        return ([],P)
    P1 = reduire(P)
    A = [0 for _ in range(n-m)]+[(-P1[n]/Q[m])*Q[i] for i in range(m)]+[-P1[n]]
    nouv_P = somme(P,A)
    U,V = quotient_reste(nouv_P,Q)
    return somme(U,[0 for _ in range(n-m)]+[P1[n]/Q[m]]), V
```